

Sisteme de Operare. Laborator 3

Honorius Gâlmeanu
galmeanu@unitbv.ro

March 5, 2023

1 Introducere

În această lucrare de laborator se descriu funcțiile I/O nebufferate pentru fișiere; operațiile I/O nebufferate nu realizează o bufferare implicită în procesul utilizator. Operațiile de scriere și citire invocă un singur apel sistem. Funcțiile definite în header-ul `stdio.h` realizează însă operații I/O bufferate.

Fișierele deschise sunt referite prin intermediul descriptorilor de fișier. Aceștia sunt reprezentați de întregi pozitivi. La deschiderea unui fișier sau la crearea unui nou, apelul de sistem returnează procesului un descriptor de fișier. Astfel vom putea referi fișierul respectiv prin descriptorul asociat.

Prin convenție, shell-ul folosește în sistemele UNIX asociază descriptorul 0 cu standard input, descriptorul 1 cu standard output iar descriptorul 2 cu standard error. Ele sunt denumite ca `STDIN_FILENO`, `STDOUT_FILENO` și `STDERR_FILENO` pentru aplicațiile ce respectă standardul POSIX. Ele sunt definite în header-ul `unistd.h`. Valoarea maximă a descriptorilor de fișier pentru un proces este dată de `OPEN_MAX`.

2 Funcții de I/O pentru fișiere

2.1 Funcția open

Este folosită pentru asocierea unui descriptor de fișier cu informațiile din File Control Block din filesystem pentru acel fișier; numărul este folosit ulterior pentru operații cu acel fișier.

```
#include <fcntl.h>
```

```
int open(const char *pathname, int flags);  
int open(const char *pathname, int flags, mode_t mode);
```

Returnează descriptorul de fișier sau -1 dacă a apărut o eroare. Argumentul `pathname` reprezintă numele fișierului ce va fi deschis sau creat. Argumentul `flags` reprezintă un **sau** logic dintre următoarele constante:

- `O_RDONLY` - deschidere pentru citire;
- `O_WRONLY` - deschidere pentru scriere;
- `O_RDWR` - deschidere atât pentru citire cât și pentru scriere.

Se va specifica obligatoriu una dintre constantele de mai sus.

- `O_APPEND` - scrierea se face doar la sfârșitul fișierului;
- `O_CREAT` - creează fișierul dacă acesta nu există. Opțiunea necesită specificarea opțiunii `mode`, biții de acces ai fișierului;
- `O_EXCL` - generează eroare dacă `O_CREAT` a fost specificată iar fișierul respectiv există;
- `O_TRUNC` - resetează lungimea fișierului la 0 dacă modul pentru care a fost deschis include scrierea;
- `O_NOCTTY` - dacă calea fișierului specifică un terminal de acces (tty), nu va aloca acel terminal ca fiind unul de control pentru proces;

- `O_NONBLOCK` - dacă calea fișierului reprezintă un fișier de tip bloc, caracter sau FIFO (pipe), va seta modul neblocaant pentru operațiile de I/O cu acest fișier;
- `O_SYNC` - fiecare `write` va aștepta terminarea operației.

Descriptorul întors de `open` este cel mai mic descriptor neutilizat disponibil.

2.2 Funcția `creat`

```
#include <fcntl.h>
```

```
int creat(const char *pathname, mode_t mode);
```

Returnează descriptorul fișierului creat sau -1 în cazul unei erori. Funcția este echivalentă cu apelul:

```
open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode);
```

2.3 Funcția `close`

```
#include <unistd.h>
```

```
int close(int fd);
```

Realizează închiderea fișierului, returnând 0 în caz de succes și -1 pentru eroare. La terminarea unui proces, se apelează de către kernel această funcție pentru toate fișierele rămase deschise.

2.4 Funcția `lseek`

Fiecare fișier deschis are un file pointer reprezentând locația unde se citește/scrie. La deschiderea fișierului locația implicită este 0. În modul `O_APPEND` acest file pointer va lua doar valori crescătoare.

```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

Returnează noul deplasament sau -1 în caz de eroare. Parametrul `whence` poate lua una dintre următoarele valori:

- pentru `SEEK_SET`, pointer-ul se setează la `offset` bytes față de începutul fișierului;
- pentru `SEEK_CUR`, pointer-ul se setează la `offset` bytes față de poziția actuală; valoarea poate fi și negativă;
- pentru `SEEK_END`, pointer-ul se setează la `offset` bytes față de sfârșitul fișierului; valoarea poate fi și negativă.

Pentru a afla poziția curentă în fișier, se folosește apelul:

```
curposs = lseek(fd, 0, SEEK_CUR);
```

Se poate seta poziția curentă în fișier mai departe de sfârșitul fișierului; se poate astfel crea un fișier cu "găuri", zone completate implicit cu bytes cu valoarea 0.

2.5 Funcția `read`

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

Citește date dintr-un fișier¹. Returnează numărul de bytes citați, 0 pentru sfârșit de fișier sau -1 în caz de eroare.

¹Folosiți secțiunea 2 din pagina de manual, man 2 read

2.6 Funcția write

```
#include <unistd.h>
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

Scrie date într-un fișier. Returnează numărul de bytes scriși sau -1 în caz de eroare. De regulă valoarea returnată va fi egală cu count; este mai mică în caz de eroare, în caz că dimensiunea fișierului a trecut peste quota alocată utilizatorului sau spațiul din sistemul de fișiere este insuficient.

Pentru ambele apeluri, de scriere și citire, poziția curentă în fișier este incrementată cu numărul de bytes citiți sau scriși.

3 Funcția fcntl

Realizează operații asupra proprietăților unui descriptor de fișier.

```
#include <fcntl.h>
```

```
int fcntl(int fd, int cmd, ... /* arg */ );
```

Este folosită în următoarele scopuri:

- duplicarea unui descriptor de fișier existent (cmd=F_DUPFD);
- preluarea/setarea flag-urilor descriptorului (cmd=F_GETFD sau cmd=F_SETFD);
- preluarea/setarea flag-urilor de stare ai fișierului (cmd=F_GETFL sau cmd=F_SETFL);
- preluarea/setarea ownership-ului I/O asincron (cmd=F_GETOWN sau cmd=F_SETOWN);
- preluarea/setarea de record locks (cmd=F_GETLK, cmd=F_SETLK sau cmd=F_SETLKW)

4 Funcția ioctl

Restul de operații I/O neacoperite de funcțiile anterioare se pot realiza folosind funcția:

```
#include <sys/ioctl.h>
```

```
int ioctl(int fd, unsigned long request, ...);
```

Funcția manipulează parametrii asociați cu fișierele speciale, de tip device. Operațiile realizate se împart în mai multe categorii:

- disk labels, necesită header-ul `disklabel.h`
- operații I/O pe device, cu header-ul `ioctl.h`
- operații cu banda magnetică - `mtio.h`
- operații cu socketi (conexiuni rețea, tot `ioctl.h`)
- operații cu terminalul (`ioctl.h`)

Un exemplu de utilizare a acestei funcții este efectuarea de operații cu banda magnetică (derulare, avans n blocuri, scriere n blocuri, etc.).

5 Example

Pentru a putea compila programele date ca exemplu va fi necesar ca mai întâi să se construiască biblioteca liblab3.a. Acesta se face cu comanda:

```
Command Line
```

```
make lib
```

După generarea bibliotecii liblab3.a, se compilează exemplele cu:

```
Command Line
```

```
make
```

5.1 Programul seek

Programul testează dacă putem poziționa locația de citire/scriere curentă pentru standard input. Se invocă funcția lseek. Programul se exemplifică mai întâi utilizând un fișier, apoi folosind un pipe ca intrare:

```
Command Line
```

```
[user@fedora lab3]$ ./seek < /etc/motd
seek OK
[user@fedora lab3]$ cat /etc/motd | ./seek
cannot seek
```

5.2 Programul hole

Programul creează un fișier având o "gaură" în el. Se invocă funcțiile creat, write și lseek.

```
Command Line
```

```
[user@fedora lab3]$ ./hole
[user@fedora lab3]$ ls -l file.hole
-rw-r--r--. 1 user user 50 Mar  5 22:41 file.hole
[user@fedora lab3]$ od -c file.hole
0000000  a  b  c  d  e  f  g  h  i  j  \0  \0  \0  \0  \0  \0
0000020  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0  \0
0000040  \0  \0  \0  \0  \0  \0  \0  \0  A  B  C  D  E  F  G  H
0000060  I  J
```

5.3 Programul mycat

Programul copiază standard input la standard output. Folosește funcțiile read și write. Se execută astfel:

```
Command Line
```

```
[user@fedora lab3]$ ./mycat < file.hole
```

5.4 Programul fileflags

Programul afișează flag-urile fișierului asociat unui anumit descriptor de fișier. Folosește funcția `fcntl`. Se execută astfel:

Command Line

```
[user@fedora lab3]$ ./fileflags 0 < /dev/tty
read only
[user@fedora lab3]$ ./fileflags 1 > temp.foo
[user@fedora lab3]$ cat temp.foo
write only
[user@fedora lab3]$ ./fileflags 2 2>> temp.foo
write only, append
[user@fedora lab3]$ ./fileflags 5 5<> temp.foo
read write
```

6 Exerciții

Exercițiul 1

Rulați toate exercițiile prezentate și încercați să înțelegeți cum lucrează. Vedeți paginile de manual pentru funcțiile utilizate.

Exercițiul 2

Dacă deschidem un fișier pentru read-write cu flag-ul pentru append, se poate citi din orice poziție specificată cu `lseek`? Se poate scrie peste datele existente? Scrieți un program pentru a verifica acest lucru.

Exercițiul 3

Cum se comportă comanda `mycat` în cazul unui fișier cu "găuri"? Scrieți un fișier cu două astfel de "găuri" și verificați. Scrieți apoi un program `mycat2` care să elimine "găurile".

Exercițiul 4

Programul `fileflags` folosește descriptorul de fișier transmis în linia de comandă. Primele două utilizări exemplificate folosesc redirectarea intrării respectiv a ieșirii. Încercați să vă dați seama ce fel de redirectare se realizează cu celelalte două redirectări exemplificate. Explicați și creați niște exemple de folosire.